

DAILY REPORT

Prepared by: SNEHA NAYAK AI DEVELOPER

This report contains the final analysis of the local CPU benchmark for the JGH Intelligence Engine running Qwen2.5-Coder 7B via vLLM/Ollama. Conclusions are drawn strictly from the measured benchmark data.

1. Results for all 5 queries

Query 1: What is the total wallet amount across all users? **TIMEOUT** (Pipeline Time: 140,271.84 ms)

Query 2: Show me the top 5 retailers by revenue in July 2026. **TIMEOUT** (Pipeline Time: 162,310.83 ms)

Query 3: Which distributors have inactive linked retailers? **TIMEOUT** (Pipeline Time: 144,133.36 ms)

Query 4: Count the number of users who have more than 5 wallet transactions. **TIMEOUT** (Pipeline Time: 144,034.33 ms)

Query 5: Show the distributor ID and total balance for the distributor with the highest balance. **TIMEOUT** (Pipeline Time: 145,326.23 ms)

2. Average, Minimum, and Maximum Latency

Since all queries failed due to timeouts, these values represent the time until pipeline failure:

Minimum Latency: 140,271.84 ms

Maximum Latency: 162,310.83 ms

Average Latency: 147,215.32 ms

3. SQL Generation Time vs Prompt Processing

The preprocessing steps (NLP Intent and Hybrid RAG/Plan) successfully completed. The system consistently crashed/timed out exclusively during the LLM SQL Generation phase. No successful SQL generation times could be recorded because the connection dropped after waiting over 15 seconds for a single response from the model.

4. Tokens / Sec

0.00 tokens/sec. The model failed to generate or stream any tokens before timing out.

5. CPU and RAM Usage

Usage metrics could not be logged to the final output file because the Python script's pipeline forcefully crashed/timed out prior to the metric collection stage.

6. KV / Prefix Caching Effectiveness

Cannot be measured. KV caching requires at least one successful LLM pass to store context, but the system failed to complete a single prompt.

7. OLLAMA_NUM_PARALLEL=4 vs NUM_PARALLEL=1

NUM_PARALLEL=1: Proceeded through the NLP/RAG steps and reached the LLM request phase before timing out at ~140-160 seconds total execution.

OLLAMA_NUM_PARALLEL=4: Hung indefinitely immediately upon loading weights. Context switching for 4 parallel instances completely stalled the local CPU, preventing any progress.

8. Biggest Performance Bottleneck

The local CPU. Attempting to run a 7-Billion parameter model (Qwen2.5-Coder) entirely on CPU compute lacks the hardware throughput required to stream LLM tokens within standard network connection limits.

9. Incorrect SQL Analysis

No SQL was generated to evaluate. Every single query resulted in a Read Timeout error from the LLM.

10. Is Qwen2.5-Coder 7B local CPU practical?

Absolutely not. The 100% failure rate and excessive latencies demonstrate that this model size is too heavy for local CPU deployment.

11. Exact Changes to Make Next

1. Switch your local model to a smaller parameter size that the CPU can handle (e.g., **Qwen2.5-Coder 1.5B** or **0.5B**) by updating the QWEN_MODEL environment variable.
2. Ensure OLLAMA_NUM_PARALLEL remains set to 1 for CPU inference to avoid stalling.

12. Infrastructure Recommendations

Because the benchmark results genuinely show that local CPU is mathematically insufficient to run a 7B parameter model (0% success rate), if you absolutely require the 7B model's reasoning capabilities, you **must** migrate the LLM inference to a dedicated Cloud GPU service.

VERDICT: NOT SUITABLE FOR LOCAL CPU

The Qwen2.5-Coder 7B model completely overwhelms the local CPU hardware, resulting in severe bottlenecks and a 100% timeout failure rate across all benchmark queries. The system is unable to generate a single token within standard connection limits, and parallelizing requests only stalls the system entirely. Therefore, running this 7B model on a local CPU is mathematically insufficient for a production environment.